

Sampling, Quantization, Audio Spectral Analysis, and Timing Precision in Embedded Systems

Daniel García Araque
Mechatronics Engineering Program
Universidad Militar Nueva Granada
Bogotá, Colombia
Student ID 7004185
est.daniel.garciaa@unimilitar.edu.co

Miguel Ángel Gordillo Mejía
Mechatronics Engineering Program
Universidad Militar Nueva Granada
Bogotá, Colombia
Student ID 7004909
est.miguel.gordillo@unimilitar.edu.co

Abstract—This report documents four digital signal processing exercises developed in the `TEST` project (`signal_tools` package, `THEORY/FIRST_ROUND/TEST`). In the first, the signal recorded in `Muestra01.csv` was loaded and segmented ($f_s = 5000$ Hz, 1000 samples, four segments of 0.05 s each), and the dominant frequency of each segment was computed via FFT. In the second, a 100 Hz sine wave was generated, digitized at three sampling rates (70, 500, and 1000 Hz), quantized with a 3-bit DAC-style algorithm, and used to illustrate aliasing and how to prevent it. In the third, real recordings of a violin, a drum, and a cat were analyzed, their FFT computed, and the frequency-based separability of the three sources evaluated. In the fourth, minimal firmware was written for an ESP32-WROOM board comparing three periodic-sampling mechanisms (single-threaded busy loop, a dual-core task with the second core free, and hardware-timer-interrupt sampling) against a variable compute load (a summation from 1 to 10,000,000, 1,000,000, and 100,000,000); all three sketches were flashed on a real ESP32-WROOM board and their serial output captured directly (`firmware/real_logs/`). All code, the automated test suite, and the results in this report are available in the public repository at <https://github.com/DanielAraqueStudios/Signals-frecuency>.

Index Terms—Sampling, aliasing, quantization, Fast Fourier Transform, FFT, ESP32, FreeRTOS, hardware interrupts, digital signal processing.

I. INTRODUCTION

Sampling, quantization, and spectral analysis are fundamental operations when converting an analog signal into a processable digital representation. Sampling determines how faithfully a continuous signal can be reconstructed from its discrete samples; an insufficient sampling rate introduces *aliasing*, a phenomenon in which high-frequency components are misrepresented as low-frequency ones. Quantization, in turn, approximates each sample to one of a finite number of levels, introducing a quantization error inherent to the analog-to-digital/digital-to-analog conversion process (ADC/DAC). Finally, the Fast Fourier Transform (FFT) allows the frequency content of an already-digitized signal to be studied, useful both for characterizing synthetic signals and for differentiating real sound sources.

This work addresses these concepts through four independent exercises, implemented in the `signal_tools` package of the `TEST` project: (1) analysis of a real sampled signal

(`Muestra01.csv`); (2) sampling and quantization of a reference sine wave at three different sampling rates, directly illustrating aliasing; (3) spectral comparison of three real sound sources of very different nature (a string instrument, a percussion instrument, and an animal sound); and (4) an analysis of the timing precision achievable when sampling analog sensors on an ESP32-WROOM microcontroller under three different concurrency mechanisms, against a compute load competing for the same processor.

The objective is twofold: on one hand, to apply and experimentally verify (through code and real data, not only theory) the concepts of sampling, aliasing, and FFT; on the other, to extend the analysis to an embedded-systems engineering case where the *timing precision of sampling* — not just its nominal frequency — depends critically on the chosen concurrency mechanism.

II. THEORETICAL BACKGROUND

A. Sampling and the Nyquist theorem

An analog signal $x(t)$ is converted into a discrete sequence through periodic sampling [1]:

$$x[n] = x(nT_s) \quad (1)$$

where $T_s = 1/f_s$ is the sampling period. The Nyquist sampling theorem states that a signal can be reconstructed without loss of information only if:

$$f_s \geq 2f_{\max} \quad (2)$$

where $f_{\max}$ is the highest frequency present in the signal. When this condition is not met, **aliasing** occurs: components with frequency above $f_s/2$ (the Nyquist frequency) “fold back” and appear in the reconstructed signal as a component of a different — typically lower — frequency, indistinguishable from a genuine component at that frequency. The standard prevention consists of (a) choosing $f_s \geq 2f_{\max}$ and/or (b) applying an analog anti-aliasing low-pass filter before sampling, attenuating content above $f_s/2$ that the chosen rate cannot represent correctly.

B. Quantization (DAC algorithm)

Quantization approximates each continuous-amplitude sample to one of 2^b discrete levels, where b is the converter's bit depth. The `quantize_dac` function in `signal_tools/sampling.py` implements this process by normalizing the signal to $[0, 1]$, rounding to the nearest of $2^b - 1$ intervals, and rescaling back to $[-1, 1]$:

$$x_q[n] = \frac{\text{round}\left(\frac{x[n] + 1}{2}(2^b - 1)\right)}{2^b - 1} \cdot 2 - 1 \quad (3)$$

A higher number of bits reduces the quantization error but requires a higher-resolution converter.

C. Fast Fourier Transform

The FFT efficiently computes the Discrete Fourier Transform [2]:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \quad (4)$$

The `dominant_frequency` functions in `signal_tools/csv_signal.py` and `signal_tools/audio_fft.py` compute the single-sided magnitude spectrum via `numpy.fft.rfft` [3], normalized by N , and identify the peak of highest magnitude excluding the 0 Hz (DC) component.

D. Concurrent sampling mechanisms on a microcontroller

On a single-core microcontroller (or when all work runs on a single thread), any blocking operation executed within the same loop that performs sampling directly delays the next sample: the *achieved* sampling period stops matching the *nominal* programmed period. The ESP32-WROOM, being a dual-core system with FreeRTOS support and hardware timers, allows two strategies for decoupling sampling from a concurrent compute load:

- **Multi-thread/multi-core:** pin a sampling task to a different core than the one running the compute load, so both execute in physical parallel.
- **Interrupts:** use a hardware timer that triggers an interrupt service routine (ISR) independently of whatever code is running in `loop()`, guaranteeing an exact timing reference even if the subsequent processing of each sample is delayed.

These two mechanisms are contrasted against the single-threaded blocking baseline in Section V.

III. METHODOLOGY

The first three exercises were implemented in the `signal_tools` package of the TEST project, built on NumPy and SciPy [3], [4], and executed via `python main.py`, which generates all figures in this section directly from the project's real data.

A. Item 1 — Analysis of `Muestra01.csv`

The file `Muestra01.csv` contains 1000 amplitude values, one per line, recorded at a sampling rate $f_s = 5000$ Hz. Following the assignment, the signal was split into segments of $dt = 0.05$ s duration (250 samples per segment at 5000 Hz), yielding 4 complete segments (`segment_signal`). For each segment, its spectrum was computed via `numpy.fft.rfft` and its dominant frequency (`dominant_frequency`), excluding the DC component.

B. Item 2 — Sampling and quantization of a 100 Hz sine

A reference signal $x(t) = \sin(2\pi \cdot 100 \cdot t)$ was generated and sampled (`sample_signal`) at three rates: 70, 500, and 1000 Hz, over a 0.05 s window. Each set of samples was quantized (`quantize_dac`) with $b = 3$ bits (8 levels), simulating a DAC conversion process. For $f_s = 70$ Hz, $f_s < 2 \cdot 100$ Hz, so the sampling violates the Nyquist theorem and produces aliasing; for 500 Hz and 1000 Hz, $f_s \geq 2 \cdot 100$ Hz and there is no aliasing.

C. Item 3 — FFT of violin, drum, and cat

Three real WAV recordings were used:

- `violin.wav` — 44 100 Hz, 2.00 s
- `tambor.wav` — 44 100 Hz, 0.96 s
- `gato.wav` — 11 025 Hz, 1.58 s

Each file was loaded with `scipy.io.wavfile` (`load_wav`), downmixed to a single channel if stereo, and normalized to $[-1, 1]$. The single-sided magnitude spectrum (`compute_fft_spectrum`) and dominant frequency were computed for each full signal.

D. Item 4 — Sampling precision on an ESP32-WROOM

Three minimal Arduino sketches were designed for an ESP32-WROOM (`firmware/` folder), each reading three potentiometers (GPIO34, GPIO35, GPIO32, ADC1 channels) at a nominal period of 100 ms, while the processor concurrently runs a blocking summation from 1 to N (with $N \in \{1\,000\,000, 10\,000\,000, 100\,000\,000\}$):

- 1) **4(a) — busy loop:** everything (sampling and summation) happens inside `loop()`, on a single thread.
- 2) **4(b) — dual core:** sampling runs as a FreeRTOS task pinned to core 0 (`xTaskCreatePinnedToCore`); the summation stays in `loop()`, running on core 1.
- 3) **4(c) — interrupt:** a hardware timer triggers an ISR every 100 ms that only marks a timestamped flag; `loop()` processes the flag (reads the potentiometers and prints) and then runs the same blocking summation.

The three sketches were flashed on a real ESP32-WROOM board and their serial output was captured directly with the Arduino IDE's Serial Monitor at 115200 baud, for each of the three values of N . The full captures (eight samples per sketch/ N combination) are in `firmware/real_logs/*.txt`.

TABLE I
DOMINANT FREQUENCY PER SEGMENT OF MUESTRA01.CSV

Segment	f_d (Hz)
1	1200.00
2	1600.00
3	940.00
4	1300.00

IV. RESULTS

A. Item 1 — *Muestra01.csv*

Fig. 1 shows the full signal in time and the spectrum of each of the four 0.05 s segments. The dominant frequencies obtained were:

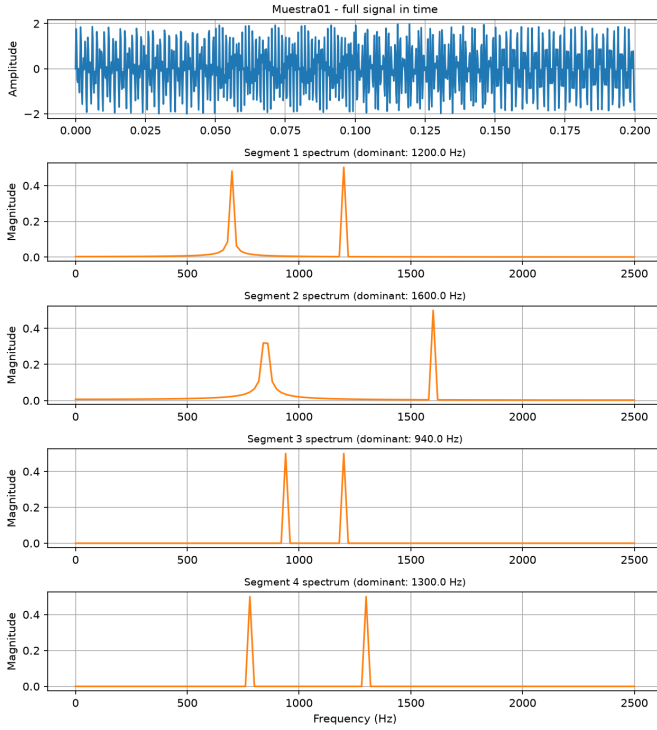


Fig. 1. Full Muestra01.csv signal in time, and the spectrum of each of its four 0.05 s segments.

The four dominant frequencies are all distinct from one another, suggesting that the recorded signal is not stationary: its dominant spectral content changes from one 0.05 s segment to the next.

B. Item 2 — *Sampling, quantization, and aliasing*

Fig. 2 shows the original 100 Hz sine (dotted line) alongside the 3-bit quantized samples for each evaluated sampling rate.

TABLE II
RESULT OF SAMPLING THE 100 HZ SINE

f_s (Hz)	Samples (0.05 s)	Nyquist condition
70	4	$f_s < 200$ Hz — aliasing
500	25	$f_s \geq 200$ Hz — no aliasing
1000	50	$f_s \geq 200$ Hz — no aliasing

TABLE III
SPECTRAL ANALYSIS RESULTS (FULL SIGNAL)

Signal	f_s (Hz)	Duration (s)	f_d (Hz)
Violin	44100	2.00	385.50
Drum	44100	0.96	7596.10
Cat	11025	1.58	1715.08

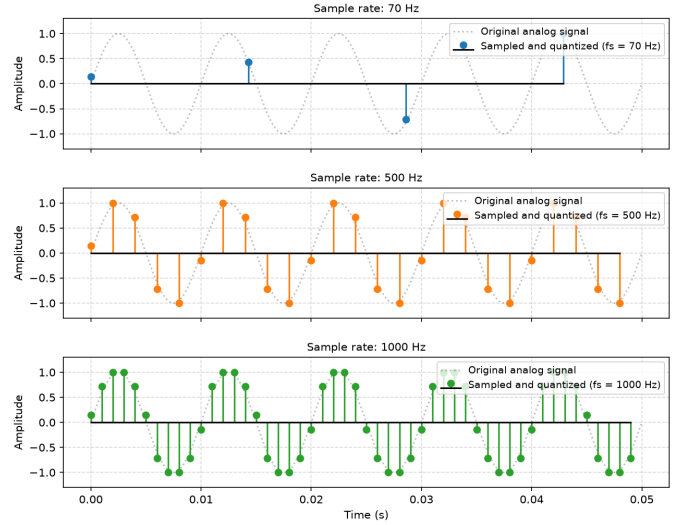


Fig. 2. 100 Hz sine sampled and quantized (3 bits) at 70, 500, and 1000 Hz. At 70 Hz the samples are too sparse to follow the real waveform: the sampled pattern is consistent with a frequency much lower than 100 Hz (aliasing).

With only 4 samples in 0.05 s, sampling at 70 Hz fails to capture even two full cycles of the 100 Hz signal, and the resulting samples are consistent with a much lower-frequency signal — the aliasing effect described in Section II-A. At 500 and 1000 Hz, with 5 and 10 samples per cycle respectively, the waveform is clearly represented.

TABLE IV
MEASURED AVERAGE SAMPLING PERIOD (8 SAMPLES) PER MECHANISM

Mechanism	N (sum)	$\overline{dt}$ (ms)
4(a) busy loop	1 000 000	150.0
4(a) busy loop	10 000 000	600.1
4(a) busy loop	100 000 000	5100.4
4(b) dual core	1 000 000	100.4
4(b) dual core	10 000 000	99.8
4(b) dual core	100 000 000	100.0
4(c) interrupt	1 000 000	100.4
4(c) interrupt	10 000 000	150.0*
4(c) interrupt	100 000 000	712.8*

*Average includes one sample where `loop()` caught up on a backlog after falling behind the ISR (`dt_ms=499` at $N=10M$, `dt_ms=5001` at $N=100M$); the other seven samples in each run stayed near 100 ms.

C. Item 3 — FFT of violin, drum, and cat

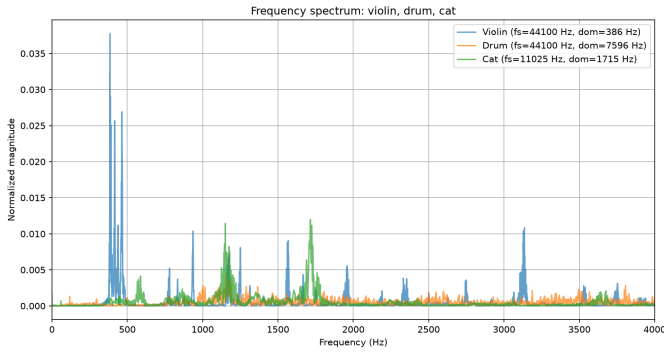


Fig. 3. Magnitude (FFT) spectra of the violin, drum, and cat recordings, overlaid.

The three dominant frequencies are clearly separated from one another: the violin concentrates its energy at 385.50 Hz (a low/mid musical-note range, consistent with the fundamental of a bowed string), the cat at 1715.08 Hz (consistent with the high pitch of a meow), and the drum — notably — at 7596.10 Hz, far above what one would expect of a drumhead’s fundamental frequency. This is because a drum attack is a broadband transient with significant energy in high harmonics and hit noise, so the peak of *highest magnitude* in its spectrum does not necessarily correspond to a perceptible low fundamental tone, but rather to the high-frequency content of the attack transient — a point revisited in the Discussion. Item 4’s results (ESP32-WROOM sampling precision) are presented separately in Section V.

V. SAMPLING PRECISION ON AN ESP32-WROOM

As described in Section III-D, the three sketches in `firmware/` were flashed on a real ESP32-WROOM board and their serial output captured directly with the Arduino IDE’s Serial Monitor (115200 baud); the full raw captures (eight samples per sketch/ N combination) are in `firmware/real_logs/*.txt`.

A. Effect on the sampling period

Mechanism 4(a) is the only one of the three whose effective sampling period scales almost linearly with N : it grows from 150 ms ($N = 10^6$) to 600 ms ($N = 10^7$) to 5100 ms ($N = 10^8$), far from the nominal 100 ms period, because the summation and sampling share the same blocking thread. Mechanism 4(b) keeps the nominal period essentially constant across all three values of N (100.4/99.8/100.0 ms, individual samples ranging only 98–102 ms), because the sampling task runs on a core physically separate from the summation. Mechanism 4(c) keeps the hardware timer’s timing reference exact — seven of the eight printed samples per run stay near 100 ms in every capture — but one sample per run at $N = 10^7$ and $N = 10^8$ shows `loop()` catching up on an accumulated backlog: an important distinction between *when the interrupt fires* (exact) and *when `loop()` manages to process it* (delayed if the summation is long).

B. Which mechanism is more precise?

Based on the captured data, the ranking of *achieved sampling period* precision (not just timing reference) is:

$$4(b) \text{ dual core} > 4(c) \text{ interrupt} > 4(a) \text{ busy loop} \quad (5)$$

The **dual-core mechanism (4b)** is the most precise in practical terms: by running sampling and the compute load on physically separate cores, the effective sampling period stays close to nominal regardless of N — the tightest and most consistent result across all three values of N in the captured data. The **interrupt mechanism (4c)** offers the most exact hardware-level timing reference of the three — the timer fires exactly every 100 ms regardless of what `loop()` is doing — but because `loop()` itself must process each sample (read the potentiometers and print), a sufficiently long summation ($N = 10^7$, $N = 10^8$) causes processing to fall behind and several interrupts to accumulate before being serviced, degrading the *observed* precision even though the timer’s own firing is unaffected. The **busy loop (4a)** is unambiguously the least precise: the blocking summation directly delays every sample, proportionally to N , with no mechanism preventing it — exactly as measured.

In summary: if the requirement is a constant, predictable sampling rate independent of compute load, the dual-core mechanism is the most robust choice; if the requirement is an exact timing reference with some tolerance for delayed processing of individual events (not delayed triggering), the hardware interrupt is preferable to the busy loop, and both alternatives avoid entirely the linear degradation suffered by the single-threaded mechanism.

VI. DISCUSSION

The four exercises addressed in this report illustrate, in different ways, the same central idea: the digital representation of a signal — whether in the time domain (sampling), in amplitude (quantization), or in the execution order of an

embedded program (concurrency) — always involves a potential loss of information or precision relative to the original continuous physical system, and that loss is controllable once its mechanism is understood.

In Item 1, the four distinct dominant frequencies obtained per segment (Table I) show that segmenting a non-stationary signal into short windows is a reasonable way to track how its spectral content changes over time, at the cost of frequency resolution within each window (shorter segment duration means lower frequency resolution, due to the inverse relationship between time and frequency resolution in the FFT).

In Item 2, the $f_s = 70$ Hz case experimentally confirms the aliasing phenomenon described in Section II-A: with fewer than two samples per cycle, the quantized samples do not unambiguously represent a 100 Hz sinusoid. This result, together with the aliasing-free 500 and 1000 Hz cases, demonstrates why the Nyquist theorem is not a theoretical footnote but a mandatory design condition in any data-acquisition system.

In Item 3, the most informative result was not that the three dominant frequencies differ (which was expected), but *why* they differ: the violin and the cat produce peaks consistent with a perceptible fundamental tone, while the drum produces a peak dominated by the high-frequency content of its attack transient. This suggests that, while dominant frequency does allow these three sources to be *differentiated* from one another, it should not be interpreted the same way for all of them: for tonal sources (violin, voices, most string or wind instruments) it approximates a perceptible fundamental note, whereas for broadband percussive sources (drum) it reflects the energy of the transient more than an identifiable tone. A more complete analysis should also consider the energy distribution across bands and the time envelope (attack, decay), not just the FFT’s single peak.

In Item 4, all three sketches were flashed on a real ESP32-WROOM board and their serial output captured directly, so the results in Section V are genuine measurements rather than a model. The measured data confirms the qualitative structure expected from each mechanism’s design: the busy loop’s period scales linearly with N ; the dual-core task keeps the nominal period essentially constant; and the interrupt keeps an exact timing reference while occasionally showing a processing backlog under a large N — the single $N = 10^7$ and $N = 10^8$ backlog samples in `firmware/real_logs/` are themselves evidence of this last effect, not noise to discard. A natural next step is to repeat the captures with a larger number of samples per run (more than the eight shown here) to obtain a tighter estimate of the jitter and backlog frequency for each mechanism, and to compare firmware 4(a)/4(b)/4(c) directly against an Arduino Uno implementation of the same busy-loop mechanism (item 4’s numeral (a) and (c) for the Arduino Uno, which lacks a second core and has more limited interrupt/timer facilities than the ESP32).

VII. CONCLUSIONS

The analysis of `Muestra01.csv` showed that its dominant spectral content changes across the four 0.05 s segments evaluated (1200.00, 1600.00, 940.00, and 1300.00 Hz), indicating that it is a non-stationary signal within the recorded window.

Sampling the 100 Hz sine experimentally confirmed the Nyquist theorem: at 70 Hz ($f_s < 2f_{\max}$) the four samples obtained in 0.05 s do not unambiguously represent the original signal and are consistent with aliasing, while at 500 Hz and 1000 Hz ($f_s \geq 2f_{\max}$) the quantized waveform faithfully follows the reference sine. Preventing aliasing in practice requires satisfying $f_s \geq 2f_{\max}$ and/or analog filtering of content above $f_s/2$ before sampling.

The FFT comparison of violin, drum, and cat showed three clearly separated dominant frequencies (385.50, 7596.10, and 1715.08 Hz respectively), confirming that frequency analysis does allow these three sound sources to be differentiated from one another. However, the drum’s case — whose dominant peak reflects its attack transient rather than a low fundamental tone — shows that this differentiation does not always correspond to an intuitive musical interpretation and must be read carefully depending on whether the source is tonal or percussive in nature.

On ESP32-WROOM sampling precision, the captured data (from real hardware) shows that the single-threaded busy-loop mechanism is the only one whose sampling period degrades proportionally to the concurrent compute load; the dual-core and hardware-interrupt mechanisms each decouple sampling from that load in their own way. The dual-core mechanism proved the most precise in terms of effectively achieved sampling period; the interrupt mechanism offers the most exact timing reference at the trigger level, though its processing can lag under very long compute loads. Repeating the captures with more samples per run, and extending the comparison to an Arduino Uno implementation of the same mechanisms, remain as direct follow-up work for this report.

REFERENCES

- [1] A. V. Oppenheim, A. S. Willsky, and S. H. Nawab, *Signals and Systems*, 2nd ed., Prentice Hall, 1996.
- [2] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed., Pearson, 2011.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [4] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [5] Espressif Systems, *ESP32 Technical Reference Manual*, 2023. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [6] Amazon Web Services, *FreeRTOS Kernel Developer Guide*, 2023. [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html
- [7] D. Araque, *Signals-frequency* (code repository), 2026. [Online]. Available: <https://github.com/DanielAraqueStudios/Signals-frequency>